

Communication Protocol

WebSocket, OSC and the C++↔React Bridge

- Message Structure

```
{ "type": "category.action", "id": "uuid", "timestamp": 0, "payload": {} }
```

- UI → Plugin Messages

TYPE	KEY PAYLOAD FIELDS
ai.prompt	prompt , provider
daw.command	command , trackId , valueDb , bpm
plugin.control *	trackId , pluginSlot , pluginName , paramName , value
plugin.query *	trackId , pluginSlot , paramName
config.set	key , value , provider
plugin.init	version , capabilities

• **Plugin → UI Messages**

TYPE	KEY PAYLOAD FIELDS	FREQUENCY
ai.stream	chunk, isDone	Per word/group
ai.response	status, content, model, durationMs	Once per prompt
daw.transport	isPlaying, bpm, positionSeconds	On change + 1Hz
daw.meter	trackId, leftDb, rightDb, peakLeftDb	~30fps
plugin.state ★	pluginName, parameters[]	On query/change
agent.memory ★	event, note	On memory update
plugin.error	code, message, severity	On error

★ = New in v1.1

• **OSC Addresses (DAW Communication)**

```
/transport/play|stop|record
/transport/bpm [float]
/track/[n]/volume|pan|mute|solo [value]
/fx/[n]/[m]/param/[p] [float]
/master/volume|gain [float]
```

• **Latency Targets**

MESSAGE TYPE	TARGET	MEASURED
WebSocket round-trip	< 5ms	2–3ms
AI TTFB (local Ollama)	< 1000ms	600–800ms
AI TTFB (cloud)	< 600ms	300–500ms
Meter update (30fps)	33ms	33ms ✓
Plugin param change	< 2ms	1ms ✓